



Язык программирования C++

Лекция 4

ПЕРЕГРУЗКА ОПЕРАТОРОВ

```
int a = 1;  
int b = 2;  
cout << a + b;
```

```
string a = "1";  
string b = "2";  
cout << a + b;
```

```
operator+(a, b)
```

```
✓ class Complex {  
  private:  
    int real, imag;  
  
  public:  
    > Complex(int r = 0, int i = 0) { ... }  
    > void print() { ... }  
  
    friend Complex operator+(Complex const& c1,  
        Complex const& c2);  
};  
✓ Complex operator+(Complex const& c1, Complex const& c2)  
  {  
    return Complex(c1.real + c2.real, c1.imag + c2.imag);  
  }
```

```
class Complex {  
private:  
    int real, imag;  
  
public:  
    Complex(int r = 0, int i = 0) { ... }  
  
    Complex operator+(Complex const& obj)  
    {  
        Complex res;  
        res.real = real + obj.real;  
        res.imag = imag + obj.imag;  
        return res;  
    }  
    void print() { ... }  
};
```

12 + i9

```
Complex c1{ 10, 5 }, c2{ 2, 4 };  
Complex c3 = c1 + c2;
```

✓ class Point

```
{  
private:  
    double m_x{};  
    double m_y{};  
    double m_z{};  
.  
public:  
    Point(double x = 0.0, double y = 0.0, double z = 0.0)  
        : m_x{ x }, m_y{ y }, m_z{ z }  
    {  
    }  
.  
    friend std::ostream& operator<< (std::ostream& out, const Point& point);  
};
```

```
const Point point{ 3.0, 4.0, 5.0 };  
std::cout << point << '\n';
```

✓ std::ostream& operator<< (std::ostream& out, const Point& point)

```
{  
    out << "Point(" << point.m_x << ", " << point.m_y << ", " << point.m_z << ')';  
    return out;  
}
```

```
Point operator- () const;  
bool operator! () const;
```

```
✓ Point Point::operator- () const  
{  
    ...  
    return Point(-m_x, -m_y, -m_z);  
}
```

```
✓ bool Point::operator! () const  
{  
    ...  
    return (m_x == 0.0 && m_y == 0.0 && m_z == 0.0);  
}
```

```
class Digit
{
private:
    int m_digit;
public:
    Digit(int digit = 0)
        : m_digit{ digit }
    {
    }
    Digit& operator++();
    Digit& operator--();
}
```

```
std::cout << digit;
std::cout << --digit;
std::cout << digit--;
```

```
Digit& Digit::operator--()
{
    if (m_digit == 0)
        m_digit = 9;
    else --m_digit;

    return *this;
}
```

```
Digit operator++(int);
Digit operator--(int);
```

```
Digit Digit::operator--(int)
{
    Digit temp{ *this };
    --(*this);
    return temp;
}
```

```
class Money
{
private:
    int m_rubles, m_kopecks;
public:
    Money(int rubles = 0, int kopecks = 0)
        : m_rubles{ rubles }, m_kopecks{ kopecks }
    {
    }

    operator int() const { return m_rubles * 100 + m_kopecks; }
};
```

```
Money money{ 12, 34 };
int tmp = money;
```

1234

```
explicit operator int() const { return m_rubles * 100 + m_kopecks; }
```

```
int tmp_im = money;
int tmp_ex = (int)money;
```


